

Product Requirements Document: Unified SOC MCP Server

Project: Unified MCP Server for SOC AI Agent

Owner: Mohammed Al3adnani

Document version: v1.0

Date: Thursday, June 11, 2026

1. Executive Summary

This project will build a single production-style Model Context Protocol server for a Security Operations Center AI agent. The server will expose structured tools for analyzing Wazuh alerts, Wazuh agents, Zeek network logs, Suricata IDS alerts, MITRE ATT&CK mappings, and correlated incident context.

The recommended architecture is Wazuh-centered. Zeek and Suricata data should be forwarded into Wazuh and queried through the Wazuh Indexer instead of requiring direct access to local Zeek or Suricata files in version 1. This approach fits the Azure deployment model and reduces the need for VPN or local tunnels.

The implementation should use the existing Wazuh MCP repository as the base because it already includes Wazuh API access, Wazuh Indexer search, authentication, retry handling, configuration loading, and structured tool modules. [[Github Repo solomonneas/wazuh-mcp](#)]

2. Problem Statement

The current project has separate MCP servers for Wazuh, Zeek, Suricata, and MITRE ATT&CK. This creates duplicated logic, inconsistent tool behavior, separate deployment units, and unnecessary complexity for an AI SOC agent.

The AI SOC agent needs one stable interface that can answer investigation questions across multiple security sources without knowing where each raw log originally came from.

The server must also avoid raw text log output, unsafe crashes, inconsistent errors, and script-like implementations.

3. Goals

- Build one unified MCP server for SOC AI workflows.
- Expose clean JSON-only tools for AI agent consumption.
- Use Wazuh as the central security data layer for alerts, agents, and indexed telemetry.
- Support Zeek-over-Wazuh and Suricata-over-Wazuh query tools.
- Map alerts and events to MITRE ATT&CK tactics and techniques.
- Correlate related evidence by IP address, time window, agent, rule, severity, and source.
- Keep the architecture modular, extensible, and safe.
- Prepare the system for Azure deployment.

4. Non-Goals

- Version 1 will not directly parse local Zeek log files.

- Version 1 will not directly parse local Suricata eve.json files.
- Version 1 will not execute firewall blocking actions.
- Version 1 will not restart Suricata, Zeek, pfSense, or Wazuh services.
- Version 1 will not retrieve packet capture files.
- Version 1 will not expose shell command execution.
- Version 1 will not create destructive Wazuh, pfSense, Zeek, or Suricata changes.

5. Target Users

- Primary user: AI SOC agent that needs structured security context.
- Secondary user: SOC analyst reviewing AI-generated incident summaries.
- Project user: Graduation project evaluator reviewing architecture, implementation quality, and demo workflows.

6. Success Criteria

- The AI agent can retrieve recent Wazuh alerts through one MCP server.
- The AI agent can query Zeek and Suricata data through Wazuh-indexed events.
- Every tool returns valid JSON with a consistent response envelope.
- Every tool catches errors and returns safe structured errors.
- No tool returns raw log lines as unstructured text.
- No tool crashes the MCP server when Wazuh, the indexer, or a query fails.
- The server runs locally and in Azure using environment variables.
- The first demo scenario can find high-severity alerts, map them to MITRE, correlate context, and return an incident summary.

7. Source Repository Strategy

Repository	Role in Unified Server	Reuse	Avoid in Version 1
wazuh-mcp	Base implementation	Wazuh API client, Indexer client, config, retries, output formatting, alert and agent tools	Major rewrite is not needed
zeek-mcp	Reusable analytics and query patterns	Aggregation, CIDR filtering, anomaly and beaconing logic, modular tool structure	Direct local Zeek file parsing and sensor commands
suricata-mcp	Reusable IDS analytics	DGA, beaconing, exfiltration, lateral movement, top-N and timeline logic	EVE file parser, PCAP replay, Suricata rule mutation, Unix socket commands
mitre-mcp	MITRE mapping and enrichment	ATT&CK data store, Wazuh alert mapping, confidence scoring, rule group mapping	Separate SOC integrations not needed in version 1

The Wazuh MCP repository already includes the Wazuh REST API client and Indexer client for searching

Wazuh alert indices. [[Github Repo solomonneas/wazuh-mcp](#)]

The Zeek MCP repository contains reusable query, aggregation, CIDR filtering, and analytics patterns that can be adapted to Wazuh-indexed Zeek events. [[Github Repo solomonneas/zeek-mcp](#)]

The Suricata MCP repository contains reusable detection analytics such as beaconing, DGA, exfiltration, lateral movement, timeline, and top-N logic. [[Github Repo solomonneas/suricata-mcp](#)]

The MITRE MCP repository contains reusable Wazuh-specific mapping logic, ATT&CK data store patterns, and confidence scoring. [[Github Repo solomonneas/mitre-mcp](#)]

8. High-Level Architecture

9. System Design Principles

- Single interface: The AI agent should call one MCP server.
- Wazuh-centered data model: Wazuh and Wazuh Indexer should be the main source for version 1.
- Modular tools: Each source should have its own tool module.
- Safe-by-default: All tools must catch errors and return structured errors.
- JSON-only: All tools must return structured JSON.
- No raw logs: Raw log content must be normalized before returning.
- Read-only first: Version 1 should analyze and correlate, not modify systems.
- Minimal dependencies: Use only necessary packages.
- Azure-ready: The server should run with Docker and environment variables.

10. Unified Response Envelope

Every tool must return this shape:

Error response shape:

11. Tool Design Rules

- Every tool must return JSON only.
- Every tool must return the unified response envelope.
- Every tool must catch exceptions.
- Every tool must sanitize errors.
- Every query tool must enforce a limit.
- Every time-based query tool must require or default a time range.
- Every tool must normalize logs into structured fields.
- No tool may return unstructured raw log text.
- No tool may expose credentials.
- No tool may expose local filesystem paths.
- No version 1 tool may execute operating system commands.
- No version 1 tool may perform destructive actions.

12. Functional Requirements

12.1 Wazuh Alert Tools

Tool	Description	Required Inputs	Output
wazuh_get_alerts	Return recent Wazuh alerts	timeRange, limit	Array of normalized alerts
wazuh_search_alerts	Search Wazuh alerts by keyword, severity, agent, rule, IP	timeRange, query fields	Array of normalized alerts
wazuh_get_alert_by_id	Return one alert by ID	alertId	Normalized alert detail
wazuh_get_high_severity_alerts	Return high and critical alerts	timeRange, severityMin, limit	Array of normalized alerts

12.2 Wazuh Agent Tools

Tool	Description	Required Inputs	Output
wazuh_list_agents	List agents and health status	optional filters	Array of agents
wazuh_get_agent_status	Return one agent status	agentId	Agent status object
wazuh_get_agent_summary	Return agent OS, IPs, status, last seen	agentId	Agent summary object

12.3 Zeek-over-Wazuh Tools

Tool	Description	Required Inputs	Output
zeek_search_logs	Search normalized Zeek events stored in Wazuh Indexer	timeRange, logType, filters	Array of Zeek events
zeek_get_dns_activity	Return DNS activity by query, source IP, or answer	timeRange, filters	Array of DNS events
zeek_get_connection_activity	Return connection activity by IP, protocol, or service	timeRange, filters	Array of connection events
zeek_detect_beaconing	Detect periodic outbound activity	timeRange, source IP, destination IP	Beaconing findings

12.4 Suricata-over-Wazuh Tools

Tool	Description	Required Inputs	Output
------	-------------	-----------------	--------

suricata_get_alerts	Return Suricata alerts indexed by Wazuh	timeRange, filters	Array of Suricata alerts
suricata_get_top_signatures	Group Suricata alerts by signature	timeRange, limit	Signature counts
suricata_get_top_talkers	Group Suricata alerts by source and destination IP	timeRange, limit	Top IP pairs
suricata_timeline	Bucket Suricata alerts over time	timeRange, bucketSize	Timeline buckets

12.5 MITRE ATT&CK Tools

Tool	Description	Required Inputs	Output
mitre_map_alert	Map an alert to ATT&CK tactics and techniques	alert object or alertId	MITRE mapping object
mitre_get_technique	Return technique metadata	techniqueId	Technique detail
mitre_rule_coverage	Analyze Wazuh rule MITRE coverage	optional rule filters	Coverage summary

12.6 Correlation Tools

Tool	Description	Required Inputs	Output
correlate_ip_activity	Correlate Wazuh, Zeek, Suricata, and pfSense activity for one IP	ip, timeRange	Timeline and findings
correlate_alert_context	Find related events around one alert	alertId, windowMinutes	Related evidence
summarize_incident	Return structured incident summary from alerts and evidence	alertIds or entity filters	Incident summary JSON

12.7 Enrichment Tools

Tool	Description	Required Inputs	Output
enrich_indicator	Normalize and enrich IP, domain, hash, URL, or user	indicator value and type	Indicator context
health_check	Check MCP, Wazuh API, and Indexer connectivity	none	Health status object

13. Data Models

13.1 Normalized Alert

13.2 Normalized Zeek Event

13.3 Normalized Suricata Alert

13.4 MITRE Mapping

13.5 Correlation Result

14. Non-Functional Requirements

- Reliability: A failing Wazuh API or Indexer request must not crash the MCP server.
- Security: Credentials must come from environment variables and must never appear in tool responses.
- Performance: Default query limit should be 50 and maximum limit should be 500.
- Maintainability: Each tool module should be separate and testable.
- Extensibility: New tools should reuse clients, query builder, response wrapper, and shared types.
- Observability: Server logs should include tool name, success/failure, duration, and sanitized error code.
- Portability: The server should run locally, in Docker, and on Azure.

15. Security Requirements

- Use TLS for Wazuh API and Wazuh Indexer connections.
- Support TLS verification configuration, but default verification should be enabled.
- Sanitize all error messages before returning them.
- Never return passwords, tokens, Authorization headers, or connection strings.
- Block command execution tools in version 1.
- Block destructive tools in version 1.
- Validate all input with schemas.
- Enforce query limits.
- Reject invalid IP, domain, hash, and technique identifiers.

16. Configuration Requirements

17. Recommended Technology Stack

Layer	Choice	Reason
Runtime	Node.js 20+	Matches the existing MCP repositories
Language	TypeScript	Strong typing and maintainability
MCP SDK	@modelcontextprotocol/sdk	Standard MCP implementation
Validation	Zod	Schema validation for tool inputs
HTTP client	Undici or built-in fetch	Minimal dependency HTTP requests
Config	dotenv plus typed env parser	Simple local and Azure configuration

Logging	pino optional	Structured logs
Deployment	Docker	Azure-friendly deployment

18. Proposed Code Structure

19. Implementation Milestones

Milestone	Scope	Exit Criteria
M1: Project foundation	TypeScript project, MCP server, config, safe response wrapper, health check	Server starts and health_check returns JSON
M2: Wazuh connectivity	Wazuh API client and Indexer client	Server can authenticate and query recent alerts
M3: Wazuh tools	Alerts and agents tools	AI can list agents and search alerts
M4: Zeek-over-Wazuh	Zeek DNS and connection tools	AI can query Zeek events from Wazuh Indexer
M5: Suricata-over-Wazuh	Suricata alert and top signature tools	AI can query and group Suricata alerts
M6: MITRE mapping	MITRE mapping module	Alerts return tactics and techniques
M7: Correlation	IP and alert context correlation	AI can produce evidence timeline
M8: Incident summary	Structured incident summary tool	AI receives ready-to-summarize incident JSON
M9: Docker and Azure	Dockerfile, env example, deployment docs	Server runs on Azure environment

20. Demo Scenarios

20.1 High-Severity Alert Investigation

- AI asks for high-severity alerts in the last 24 hours.
- MCP queries Wazuh Indexer.
- MCP normalizes alerts.
- MCP maps alerts to MITRE ATT&CK.
- MCP returns JSON only.
- AI explains the incident using the structured result.

20.2 Suspicious IP Correlation

- AI asks for all activity related to one IP.
- MCP queries Wazuh alerts, Zeek events, Suricata alerts, and pfSense events.
- MCP builds a timeline.

- MCP returns findings with evidence count and severity.

20.3 Suricata Signature Review

- AI asks for top Suricata signatures in a time range.
- MCP groups alerts by signature ID and category.
- MCP returns top signatures and affected hosts.

20.4 Zeek DNS Investigation

- AI asks for suspicious DNS activity.
- MCP queries Zeek DNS events from Wazuh Indexer.
- MCP applies optional DGA or entropy analysis.
- MCP returns structured DNS findings.

21. Acceptance Criteria

- All tools return the unified JSON response envelope.
- All tools validate inputs.
- All tools handle errors safely.
- All tools enforce limits.
- All tools avoid raw unstructured logs.
- Wazuh alert search works.
- Wazuh agent listing works.
- Zeek-over-Wazuh DNS or connection query works.
- Suricata-over-Wazuh alert query works.
- MITRE mapping works for alerts with Wazuh MITRE fields.
- MITRE fallback mapping works when Wazuh MITRE fields are missing.
- IP correlation returns a timeline and findings.
- Docker deployment works.
- README explains local and Azure setup.

22. Risks and Mitigations

Risk	Impact	Mitigation
Zeek or Suricata logs are not correctly ingested into Wazuh	Zeek-over-Wazuh and Suricata-over-Wazuh tools return empty results	Start with Wazuh alerts, then validate index fields before building tools
Wazuh Indexer field names differ by decoder	Queries may miss data	Create normalization module and configurable field mappings
Large queries are slow	Bad AI user experience	Require time range and limit every query
MITRE fields are missing	Mapping quality decreases	Use fallback mapping by rule ID, groups, and keywords

Credentials leak through errors	Security issue	Centralize error sanitization and test redaction
Scope grows too fast	Project becomes hard to finish	Keep version 1 read-only and Wazuh-centered

23. Open Questions

- What exact Wazuh Indexer fields contain Zeek log type and Suricata event type after ingestion?
- Will pfSense logs also be forwarded into Wazuh for version 1?
- Will the AI agent connect through stdio, HTTP, or another MCP transport?
- Will n8n be used only for notifications or also for response workflows?
- Do you need Azure Container Apps, Azure VM, or Docker Compose on a VM?

24. Development Roadmap

Phase 1: Foundation

- Create TypeScript project.
- Add MCP server startup.
- Add environment config.
- Add response envelope.
- Add safe tool wrapper.
- Add health_check tool.

Phase 2: Wazuh Core

- Add Wazuh API client.
- Add Wazuh Indexer client.
- Add alert normalization.
- Add agent normalization.
- Add Wazuh alert tools.
- Add Wazuh agent tools.

Phase 3: Network Security Sources

- Add Zeek field mapping.
- Add Zeek DNS tool.
- Add Zeek connection tool.
- Add Suricata field mapping.
- Add Suricata alert tool.
- Add Suricata top signatures tool.

Phase 4: Intelligence and Correlation

- Add MITRE mapper.

- Add MITRE technique lookup.
- Add IP correlation.
- Add alert context correlation.
- Add incident summary output.

Phase 5: Deployment and Demo

- Add Dockerfile.
- Add docker-compose.yml.
- Add .env.example.
- Add Azure deployment notes.
- Add demo scenarios.
- Add troubleshooting guide.

25. Definition of Done

- The unified MCP server starts successfully.
- The health check validates Wazuh API and Indexer connectivity.
- At least 10 tools are implemented with consistent JSON responses.
- All implemented tools use safe error handling.
- All implemented tools validate inputs.
- All implemented tools have structured output types.
- The server can be deployed with Docker.
- The README documents setup, environment variables, and demo usage.
- The graduation demo can show alert retrieval, MITRE mapping, and correlation.

Product Requirements Document: AI SOC Agent

Project: AI SOC Agent for Unified SOC MCP Server

Owner: Mohammed Al3adnani

Document version: v1.0

Date: Thursday, June 11, 2026

1. Executive Summary

This product will build an AI Security Operations Center agent that uses the Unified SOC MCP Server to investigate Wazuh alerts, Zeek network events, Suricata IDS alerts, MITRE ATT&CK mappings, and correlated incident evidence.

The agent will not directly parse raw logs or connect separately to Zeek, Suricata, or pfSense in version 1. It will use the unified MCP server as its only security-data interface.

The agent will help a SOC analyst ask security questions, investigate alerts, correlate evidence, map

activity to MITRE ATT&CK, and generate structured incident summaries.

2. Problem Statement

Security data from Wazuh, Zeek, Suricata, pfSense, and MITRE ATT&CK can be difficult to analyze manually because each source has different fields, terminology, severity models, and investigation workflows.

The Unified SOC MCP Server solves the backend tool layer, but the project still needs an AI agent that knows how to use those tools safely and consistently.

The AI agent must avoid hallucinated security conclusions, avoid destructive actions in version 1, and only reason from structured tool results.

3. Goals

- Build an AI SOC analyst agent that communicates with the Unified SOC MCP Server.
- Support alert triage, investigation, correlation, MITRE mapping, and incident summarization.
- Use structured JSON tool results as the source of truth.
- Ask follow-up questions when required investigation inputs are missing.
- Return clear security explanations for a human analyst.
- Produce consistent incident summaries with evidence, severity, timeline, affected assets, MITRE mapping, and recommended next steps.
- Keep version 1 read-only and safe.

4. Non-Goals

- Version 1 will not automatically block IP addresses.
- Version 1 will not modify pfSense firewall rules.
- Version 1 will not restart Zeek, Suricata, Wazuh, or pfSense services.
- Version 1 will not execute shell commands.
- Version 1 will not ingest raw logs directly.
- Version 1 will not train a custom model.
- Version 1 will not replace a human SOC analyst for final incident decisions.

5. Target Users

- Primary user: SOC analyst who needs fast investigation support.
- Secondary user: Student project evaluator reviewing the AI workflow and architecture.
- System user: Unified SOC MCP Server that provides structured security tools.

6. Product Scope

The AI SOC Agent will operate as a reasoning and orchestration layer above the Unified SOC MCP Server.

The agent will receive a user question, decide which MCP tools are needed, call those tools, analyze structured JSON responses, and return a security-focused answer.

The agent will use the MCP server as the only backend integration point for Wazuh, Zeek, Suricata, MITRE

ATT&CK, and incident correlation.

7. High-Level Architecture

8. Agent Responsibilities

Responsibility	Description	Example
Alert triage	Find and prioritize alerts by severity, time range, agent, and rule.	Find high-severity alerts in the last 24 hours.
Evidence collection	Collect related Zeek, Suricata, Wazuh, and pfSense events.	Find all activity for a suspicious IP.
MITRE mapping	Explain tactics and techniques linked to an alert.	Map this brute-force alert to ATT&CK.
Correlation	Build timelines from multiple evidence sources.	Correlate this alert with DNS and IDS activity.
Incident summarization	Create a structured incident report from evidence.	Summarize what happened and what to do next.
Analyst guidance	Recommend safe next investigation steps.	Check affected host, user, source IP, and related alerts.

9. Agent Design Principles

- Tool-first reasoning: The agent must use MCP tool results before making security claims.
- Evidence-based answers: Every conclusion should reference evidence from structured results.
- Read-only by default: Version 1 must not perform destructive actions.
- Structured outputs: Incident results should follow predictable sections.
- Safe uncertainty: The agent should say when evidence is insufficient.
- Explainability: The agent should explain why an alert matters.
- Minimal assumptions: The agent should not invent fields, alerts, hosts, users, or timelines.

10. Core Agent Modes

Mode	Purpose	Typical Tools
Triage Mode	Identify important alerts quickly.	wazuh_get_high_severity_alerts, wazuh_search_alerts
Investigation Mode	Collect evidence around an alert, IP, host, user, or rule.	correlate_ip_activity, correlate_alert_context
Threat Mapping Mode	Map alerts to MITRE tactics and techniques.	mitre_map_alert, mitre_get_technique
Network Analysis Mode	Review Zeek and Suricata network evidence.	zeek_get_dns_activity, zeek_get_connection_activity, suricata_get_alerts
Reporting Mode	Generate incident summaries for analyst review.	summarize_incident, correlate_alert_context

11. Functional Requirements

11.1 Alert Triage

- The agent must retrieve recent high-severity alerts.
- The agent must explain alert severity using rule level, source, affected agent, and MITRE context when available.
- The agent must group related alerts when they involve the same IP, agent, rule, or time window.
- The agent must identify missing context and request additional data if needed.

11.2 Alert Investigation

- The agent must investigate one alert by ID.
- The agent must collect related events before and after the alert timestamp.
- The agent must build a timeline of related events.
- The agent must separate confirmed evidence from assumptions.

11.3 IP Investigation

- The agent must investigate one IP address across Wazuh, Zeek, Suricata, and pfSense data.
- The agent must classify the IP as internal, external, unknown, or private when possible.
- The agent must identify source and destination roles when evidence supports it.
- The agent must summarize related alerts, DNS activity, connections, and IDS signatures.

11.4 MITRE Explanation

- The agent must explain mapped MITRE tactics and techniques in analyst-friendly language.
- The agent must prefer Wazuh rule.mitre fields when available.
- The agent must use fallback mapping only when direct MITRE fields are missing.
- The agent must include confidence when mapping is uncertain.

11.5 Incident Summary

- The agent must generate structured incident summaries.
- The summary must include title, severity, status, affected assets, timeline, evidence, MITRE mapping, confidence, and recommended next steps.
- The summary must not claim containment, eradication, or remediation unless tool evidence confirms it.

12. Agent Output Format

The agent should use this structure for investigation answers:

13. Agent Safety Rules

- The agent must not invent alerts, IPs, hostnames, users, domains, hashes, or timestamps.
- The agent must not call destructive tools in version 1.
- The agent must not recommend blocking unless evidence supports malicious or suspicious activity.

- The agent must distinguish between suspicious, confirmed malicious, and benign activity.
- The agent must not expose credentials, tokens, API keys, or configuration secrets.
- The agent must not return raw logs to the user.
- The agent must summarize sensitive evidence into structured fields.
- The agent must ask for a time range when the user request is too broad.
- The agent must use default limits to avoid excessive queries.

14. Tool Use Policy

User Intent	Agent Action	Tool Strategy
Find recent alerts	Query Wazuh alerts.	Use time range and limit.
Investigate alert ID	Fetch alert, map MITRE, correlate context.	Use alert lookup, mapping, correlation.
Investigate IP	Correlate across sources.	Use IP correlation and enrichment.
Explain technique	Retrieve MITRE details.	Use MITRE technique lookup.
Create report	Collect evidence first, then summarize.	Use incident summary after evidence collection.
Take response action	Refuse automatic destructive action in version 1.	Offer safe manual steps or draft workflow.

15. Prompt and Instruction Requirements

The agent system instructions should include these rules:

- You are an AI SOC analyst assistant.
- You must use MCP tools for security facts.
- You must never invent evidence.
- You must never perform destructive actions in version 1.
- You must explain conclusions using evidence.
- You must return structured incident summaries for investigations.
- You must distinguish evidence, inference, and recommendation.
- You must ask for clarification when alert ID, IP, time range, or scope is missing.

16. Conversation Flows

16.1 High-Severity Alert Flow

1. User asks for high-severity alerts.
2. Agent chooses a default time range if the user does not provide one.
3. Agent calls Wazuh high-severity alert tool.
4. Agent maps alerts to MITRE when available.
5. Agent groups alerts by agent, source IP, rule, and technique.
6. Agent returns prioritized findings and next steps.

16.2 Alert Investigation Flow

1. User provides an alert ID.
2. Agent fetches the alert detail.
3. Agent maps the alert to MITRE.
4. Agent correlates events around the alert timestamp.
5. Agent builds a timeline.
6. Agent returns incident summary and analyst next steps.

16.3 Suspicious IP Flow

1. User provides an IP address.
2. Agent validates the IP format.
3. Agent queries correlation by IP.
4. Agent reviews Wazuh, Zeek, Suricata, and pfSense evidence.
5. Agent ranks findings by severity and confidence.
6. Agent recommends safe follow-up steps.

17. Agent Memory and Context

- The agent may remember project architecture decisions if the user explicitly asks it to remember them.
- The agent should not store credentials or secrets in memory.
- The agent should maintain investigation context within a conversation.
- The agent should reference previous alerts or entities in the same investigation when relevant.

18. Non-Functional Requirements

- Reliability: The agent must handle MCP tool errors without failing the full conversation.
- Explainability: The agent must show how it reached conclusions.
- Safety: The agent must remain read-only in version 1.
- Consistency: Similar inputs should produce similar response structure.
- Extensibility: New tools should be added through clear routing rules.
- Performance: The agent should avoid unnecessary tool calls and use targeted queries.

19. Agent Deployment Options

Option	Description	Best Use
n8n AI Agent	Agent workflow built in n8n with MCP/tool integration.	Fast demo and visual automation.
Custom Python Agent	Python app that connects to MCP server and LLM API.	More control over orchestration.
Custom TypeScript Agent	Node.js app using TypeScript and MCP client libraries.	Same language as MCP server.
Desktop MCP Client	Local agent client connected to MCP server.	Prototype and testing.

For the graduation project, the recommended path is a custom TypeScript or Python agent for control, plus optional n8n workflows for notifications and automation.

20. Recommended Version 1 Agent Capabilities

- Investigate recent high-severity alerts.
- Investigate one alert by ID.
- Investigate one IP address.
- Explain MITRE tactics and techniques for an alert.
- Summarize incident evidence.
- Generate analyst next steps.
- Trigger optional n8n notification workflow only after human approval.

21. Future Version 2 Capabilities

- Human-approved IP blocking workflow.
- Human-approved pfSense rule-change workflow.
- Case creation in a case-management system.
- Threat intelligence integrations.
- Automated recurring alert triage.
- Multi-incident dashboard.
- Natural-language report export.

22. Acceptance Criteria

- The agent can answer: find high-severity alerts from the last 24 hours.
- The agent can answer: investigate this alert ID.
- The agent can answer: correlate activity for this IP address.
- The agent can explain MITRE mapping for an alert.
- The agent can produce a structured incident summary.
- The agent refuses or safely defers destructive actions in version 1.
- The agent clearly states limitations when tool data is missing.
- The agent uses MCP tool evidence before giving security conclusions.

23. Risks and Mitigations

Risk	Impact	Mitigation
Agent hallucinates conclusions	Bad security guidance	Require evidence-first tool use and limitations section.
Tool results are incomplete	Incorrect investigation summary	Show missing data and ask follow-up questions.
User asks for destructive action	Operational risk	Keep version 1 read-only and require human approval later.
Too many tool calls	Slow response	Use scoped workflows and default limits.

MITRE mapping is uncertain	Misleading explanation	Return confidence and mapping reason.
----------------------------	------------------------	---------------------------------------

24. Implementation Roadmap

Phase 1: Agent Foundation

- Create agent runtime.
- Add MCP client connection.
- Add base system instructions.
- Add structured output formatter.
- Add error and limitation handling.

Phase 2: Triage Workflows

- Add high-severity alert triage workflow.
- Add alert search workflow.
- Add alert explanation workflow.

Phase 3: Investigation Workflows

- Add alert ID investigation workflow.
- Add IP investigation workflow.
- Add timeline generation.
- Add evidence grouping.

Phase 4: Intelligence Workflows

- Add MITRE explanation workflow.
- Add confidence scoring display.
- Add recommended next-step generation.

Phase 5: Reporting and Demo

- Add incident summary workflow.
- Add graduation demo scenarios.
- Add README instructions.
- Add sample prompts and expected outputs.

25. Definition of Done

- The AI SOC Agent connects to the Unified SOC MCP Server.
- The agent can run at least five investigation workflows.
- The agent produces structured incident output.
- The agent handles tool errors safely.
- The agent does not perform destructive actions.

- The agent can support a graduation demo with Wazuh, Zeek, Suricata, MITRE, and correlation evidence.

Sources Considered

- [Wazuh MCP repository](#)
- [Zeek MCP repository](#)
- [Suricata MCP repository](#)
- [MITRE MCP repository](#)
- Conversation requirements from Mohammed Al3adnani on Thursday, June 11, 2026